Tutorial: Reverse Engineering, Part 3

Max Shapiro, Ryan Evans, Boyang Wang

Data and System Security Lab (DaSec), Department of ECE, University of Cincinnati
Last Modified: April, 2026, Status: Completed

1 Introduction

What This Document is About. This document covers creating a new project in Ghidra, navigation and comprehension of Ghidra's multi-panel user interface, going through a binary analysis overview with a simple introductory binary example compiled from a C program, and establishing the fundamentals of reverse engineering techniques (e.g., how to find a hard-coded password and how to patch binary to change the behavior of a program) in general. In addition, we also describe how to perform these steps with Binary Ninja and IDA (Free), another two popular reverse engineering tools.

Pre-Requisites. Our tutorial document, named Revere Engineering Part 1, describes how to install a Virtual Machine (VM) and Ghidra. We suggest that readers review it first. Before proceeding with our Ghidra analysis, we need to set up our example file. Please download the following code from our GitHub repository onto the installed VM with the following CLI command:

```
$ curl -O https://raw.githubusercontent.com/UCdasec/Tutorial-Ghidra/main/src/example_1_password.c
```

GitHub Repository. Our tutorial document (Part 1, Part 2, Part 3) as well as the code and tutorial videos can be found on our GitHub repository[1]. Both Part 2 and Part 3 focus on similar reverse engineering process, where Part 2 focuses on a C++ program while Part 3 is based on a C program.

[1] https://github.com/UCdasec/Tutorial-Ghidra

2 Password Example in C

To demonstrate the effective use of Ghidra, it is very important to understand the compilation process, i.e., the process of transforming high-level code (e.g., C code) into machine code. In our first example, we have a program, named `example_1_password.c`, written in C. Given this program, a user is asked to enter a password each time from the terminal until it matches the correct one ("`pass`") pre-defined in the code. When a user enters the correct password, the program calls `printf` showing "Correct" in the terminal. Otherwise, if the input is incorrect, it will print out with"Wrong" in the terminal. The code snippet is presented below.

```c
// example_1_password.c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <string.h>
#include <stdbool.h>

#define PASSWORD "pass" // pre-defined password, hard-coded
#define MAX_LENGTH 10

bool password_check(char input[]) {
    bool flag = 0;
    if (strncmp(input, PASSWORD, MAX_LENGTH) == 0) {
        flag = 1;
    }

    if (flag == 1)  {
        printf("Correct\n");
    } else {
        printf("Wrong\n");
    }
    return flag;
}

int main(){
    char input[MAX_LENGTH + 1];
    printf("Enter the password (max %d characters): ", MAX_LENGTH);
    bool flag = 0;

    if (fgets(input, sizeof(input), stdin) != NULL) {
        if (strchr(input, '\n') == NULL) {
            flag = 0;
        }
        else {
            input[strcspn(input, "\n")] = '\0';
        };
```

```
        flag = password_check(input);
    };

    return 0;
}
```

Using a compiler, such as GNU Compiler Collection (GCC), we can convert human readable code to machine instructions that our target processor can understand. Usually, most IDEs will come equipped with a compiler installed and one can generate a binary with machine instructions by simply clicking a button in an IDE to complete the process, where the IDE runs the commands behind the scene. To make the process more explainable, we compile our code directly and generate an executable file (`example_1_password`) by running the command below in the terminal:

```
$ gcc -static -o example_1_password example_1_password.c
```

This executable file is a binary file that contains all the machine instructions generated by the C program. The `-static` parameter is used to tell the compiler to insert all the libraries in the binary that will be generated. The `-o` parameter lets us name our file to `example_1_password` (-o for Output). And lastly, at the end is the name of the source code file being compiled.

It is important to understand that compiling any program is a multi-stage process with the following steps:

- **Preprocessing** – regarding C/C++, any directives that begin with # (ex. #include, #define, #ifdef, #ifndef) is processed, inserting the contents from the header files, replacing and expanding macros and constants, and provide conditional compilation respectively.
- **Compilation** – During this step the preprocessed code is translated into assembly using various theories, breaking code into tokens (Lexical analysis), check grammar and structure of the code (Syntax analysis), ensure logic and types are correct (Semantic Analysis), and lastly provides optimization to the code to improve performance. Note: any errors in code would typically be detected in this stage.
- **Assembly** – Turns assembly code into object code (machine code, binary opcodes), e.g., an object file (.o).
- **Linking** – combines multiple object files and external libraries into a complete program. This stage will output a single executable file.

When we only have a binary file, either an object file or executable file, but no access to the source code, software reverse engineering can perform the analysis on the binary and help us understand the functionalities and behavior of this binary before we use it or integrate it into a system. Ghidra offers a powerful decompiler and a range of various analyzers to predict what the source code and function may look like.

2.1 Initial Binary Analysis

Assume one has compiled the source code `example_1_password.c` and generate the binary `example_1_password` successfully or downloaded the binary directly from our GitHub repo, we can run the binary first with the command below to observe any distinctive behavior, identify any issues, and determine what actions we can take.

```
$ ./example_1_password
```

After running it, we can see that it is asking for a specific input string as a magic word. Once we type a random string, the program asks for a new input again, which indicates the magic word is a pre-defined string (e.g. a password) by the program and the one we just provide does not match to the password.

Once we have the initial attempt above with the program, we can first utilize a couple of command-line tools to obtain some basic information for reverse engineering on this program. We can first run `file <filename>`, in our case `file` is `example_1_password`, then we will see a description of that file.

```
$ file example_1_password
```

After running the file command above, we can see that the file is ELF (Executable and Linkable Format) formatted, 64-bit, x86 architecture, sha1 hash for integrity, and lastly it is not stripped (i.e., a binary with headers and debug information). This information is helpful before using Ghidra.

Fig. 1: After running command **file Ghidra_EX_1**

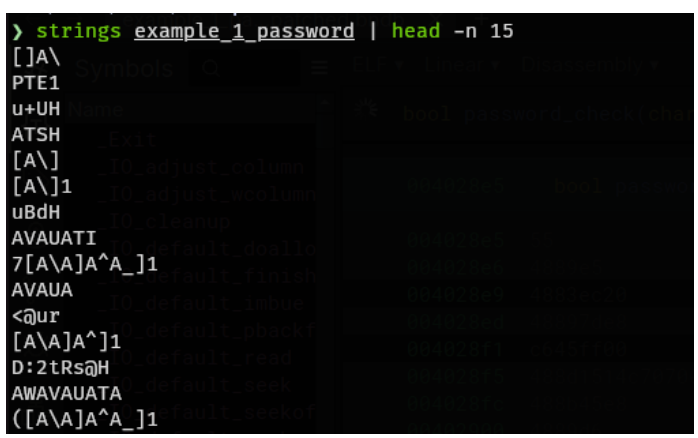


Fig. 2: After running command **strings example_1_password**

Next, we can run command `strings <filename>`, which will extract sequences of printable characters from this binary. It is helpful to detect function and variable names in the binary. One should also keep in mind that this command will also print function names from libraries, which may not be specifically useful to the target program.

```
$ strings ./example_1_password | head -n 15
```

```
❮ objdump -h example_1_password

example_1_password:     file format elf64-x86-64

Sections:
Idx Name          Size      VMA               LMA               File off  Algn
  0 .rela.plt     00000210  0000000000400270  0000000000400270  00000270  2**3
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  1 .init         0000001b  0000000000401000  0000000000401000  00001000  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, CODE
  2 .plt          000000b0  0000000000401020  0000000000401020  00001020  2**3
                  CONTENTS, ALLOC, LOAD, READONLY, CODE
  3 .text         0007dafd  0000000000401100  0000000000401100  00001100  2**6
                  CONTENTS, ALLOC, LOAD, READONLY, CODE
  4 .fini         0000000d  000000000047ec00  000000000047ec00  0007ec00  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, CODE
  5 .rodata       0001bfac  000000000047f000  000000000047f000  0007f000  2**5
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  6 rodata.cst32  00000060  000000000049afc0  000000000049afc0  0009afc0  2**5
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  7 .eh_frame     00009e6c  000000000049b020  000000000049b020  0009b020  2**3
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  8 .gcc_except_table 0000012c  00000000004a4e8c  00000000004a4e8c  000a4e8c  2**0
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  9 .note.gnu.property 00000040  00000000004a4fb8  00000000004a4fb8  000a4fb8  2**3
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
 10 .note.ABI-tag 00000020  00000000004a4ff8  00000000004a4ff8  000a4ff8  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
 11 .tdata        00000030  00000000004a6e10  00000000004a6e10  000a5e10  2**3
                  CONTENTS, ALLOC, LOAD, DATA, THREAD_LOCAL
 12 .tbss         00000028  00000000004a6e40  00000000004a6e40  000a5e40  2**3
                  ALLOC, THREAD_LOCAL
 13 .init_array   00000010  00000000004a6e40  00000000004a6e40  000a5e40  2**3
                  CONTENTS, ALLOC, LOAD, DATA
 14 .fini_array   00000010  00000000004a6e50  00000000004a6e50  000a5e50  2**3
                  CONTENTS, ALLOC, LOAD, DATA
 15 .data.rel.ro  000040e8  00000000004a6e60  00000000004a6e60  000a5e60  2**5
                  CONTENTS, ALLOC, LOAD, DATA
 16 .got          00000098  00000000004aaf48  00000000004aaf48  000a9f48  2**3
                  CONTENTS, ALLOC, LOAD, DATA
 17 .got.plt      000000c8  00000000004aafe8  00000000004aafe8  000a9fe8  2**3
                  CONTENTS, ALLOC, LOAD, DATA
 18 .data         000019e0  00000000004ab0c0  00000000004ab0c0  000aa0c0  2**5
                  CONTENTS, ALLOC, LOAD, DATA
 19 .bss          00005808  00000000004acaa0  00000000004acaa0  000abaa0  2**5
                  ALLOC
 20 .comment      00000012  0000000000000000  0000000000000000  000abaa0  2**0
                  CONTENTS, READONLY
```

Fig. 3: After running command **objdump -h example_1_password**

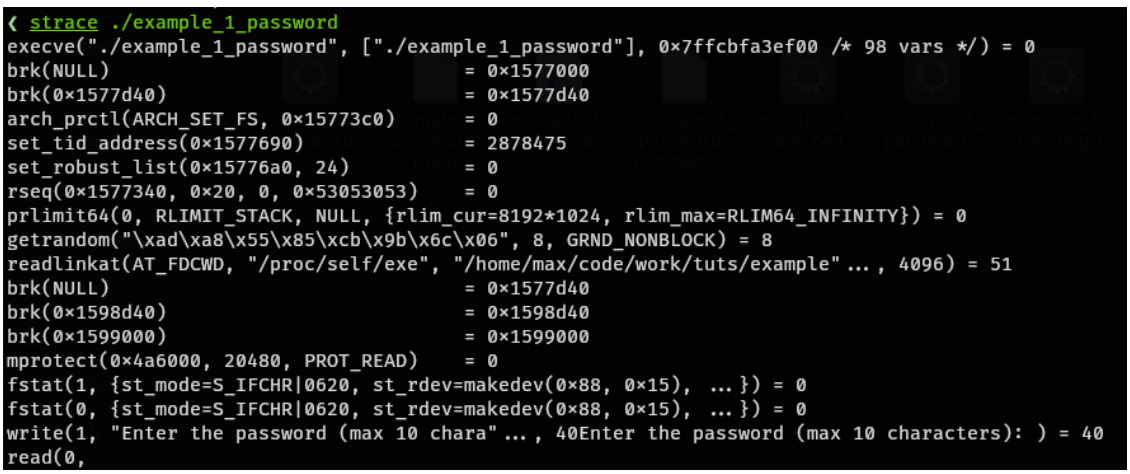
In addition, we can also run `objdump` or `hexdump`, particularly with parameters like `-h`. This command allows us to inspect section headers, raw file type, the structure of a binary. *It is worth*

mentioning that all these commands are static, i.e., we obtain the information of a binary without executing it.

```
$ objdump -h  example_1_password
```

Another useful command is `strace/ltrace`. Either command will give an outline of what system processes are called associated with a binary. Specifically, `strace` runs the program and intercepts and records all system calls made by the running program until the program is stopped. This is a dynamic tool, meaning the binary will be executed while we run this command.

Once we have obtained the basic information of a binary with the commands above, we are ready to move on to the next phase of reverse engineering. Specifically, we will utilize Ghidra to extract hard-coded secret code and manipulate the binary in a way in which any input from the user would be accepted.



```
❮ strace ./example_1_password
execve("./example_1_password", ["./example_1_password"], 0x7ffcbfa3ef00 /* 98 vars */) = 0
brk(NULL)                               = 0x1577000
brk(0x1577d40)                          = 0x1577d40
arch_prctl(ARCH_SET_FS, 0x15773c0)      = 0
set_tid_address(0x1577690)              = 2878475
set_robust_list(0x15776a0, 24)          = 0
rseq(0x1577340, 0x20, 0, 0x53053053)    = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
getrandom("\xad\xa8\x55\x85\xcb\x9b\x6c\x06", 8, GRND_NONBLOCK) = 8
readlinkat(AT_FDCWD, "/proc/self/exe", "/home/max/code/work/tuts/example"..., 4096) = 51
brk(NULL)                               = 0x1577d40
brk(0x1598d40)                          = 0x1598d40
brk(0x1599000)                          = 0x1599000
mprotect(0x4a6000, 20480, PROT_READ)    = 0
fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0x15), ...}) = 0
fstat(0, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0x15), ...}) = 0
write(1, "Enter the password (max 10 chara"..., 40Enter the password (max 10 characters): ) = 40
read(0,
```

Fig. 4: After running command **strace ./example_1_password**

3 Creating a Ghidra Project

The first window shown whenever Ghidra is opened is the project selection window. To import a binary into Ghidra, a project must be created first.

1. Press *File* → *New Project* to open the project creation wizard
2. Select *Non-Shared Project* → Next → Pick any project directory and name.

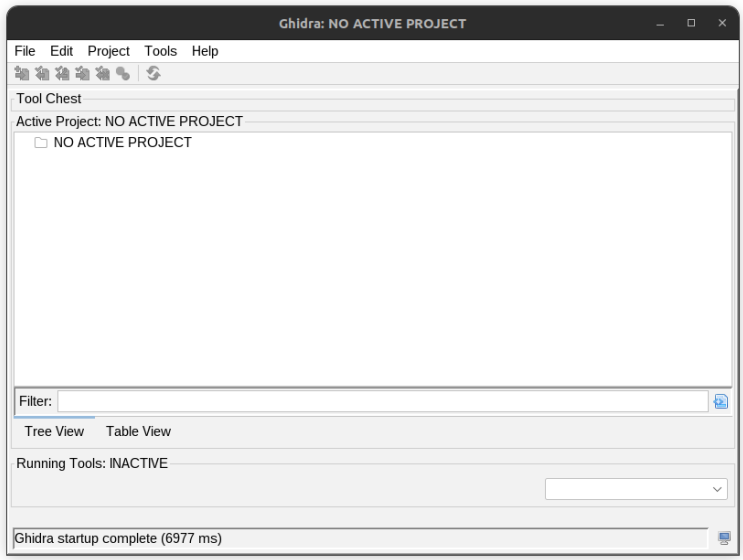


Fig. 5: First window: project selection

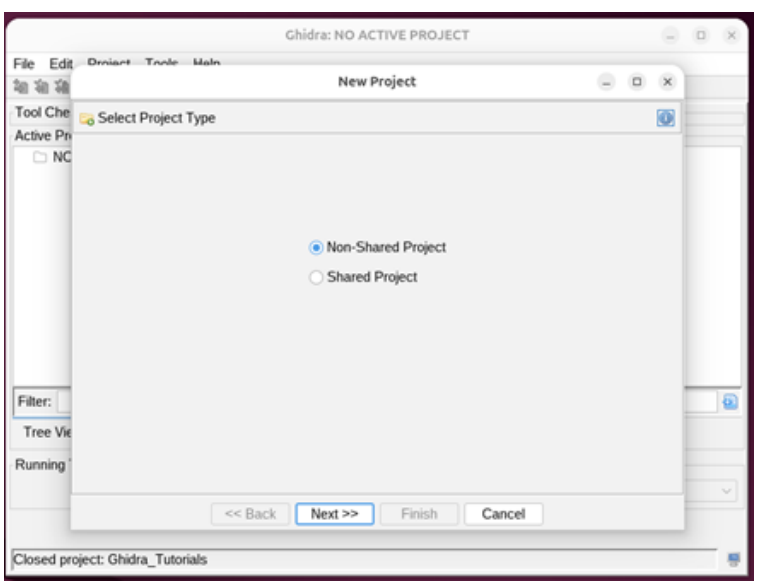


Fig. 6: New Project Window

Next, click on **File** → **Import File** and browse to the target binary (*example_1_password*). We will leave the following settings as they are since the data is accurate (as we observed by running the `file` command). Once it is completed, we can observe the summary window regarding this binary target.

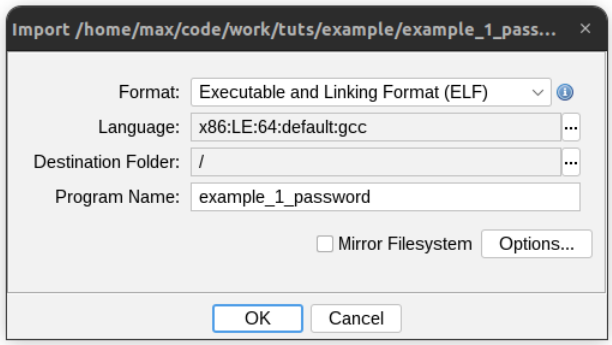


Fig. 7: Import file: *example_1_password*

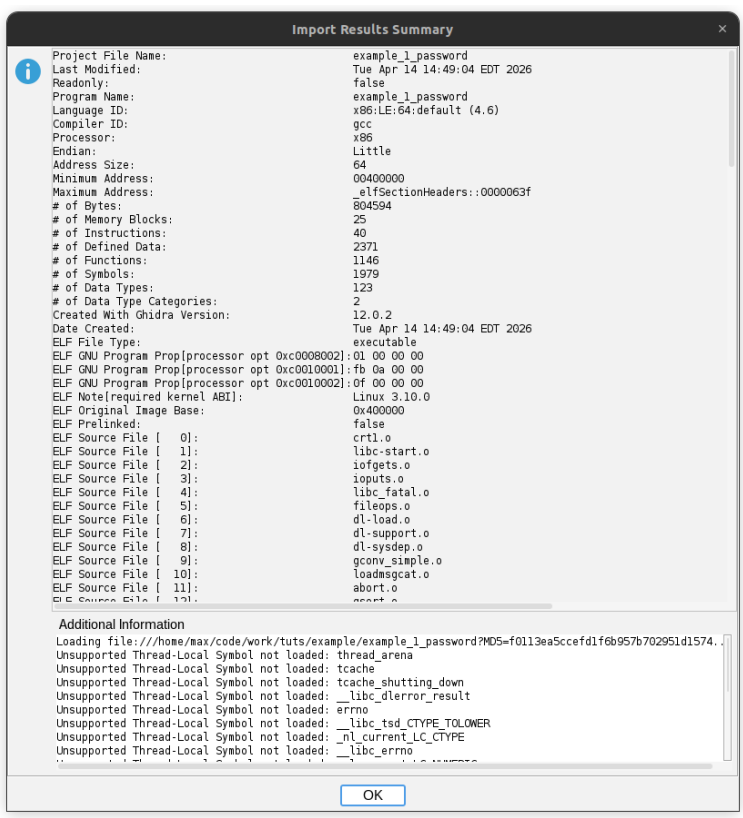


Fig. 8: Summary Window after Adding the Binary

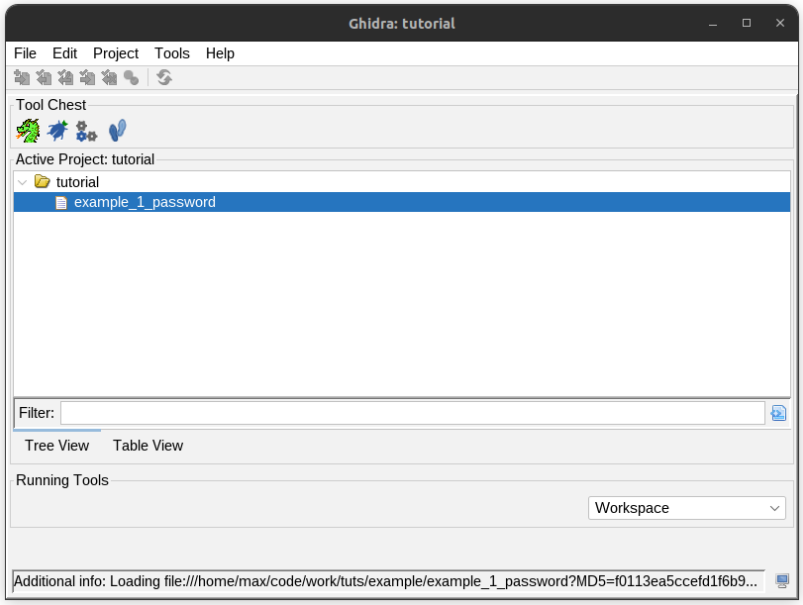


Fig. 9: Project View after Adding the Binary

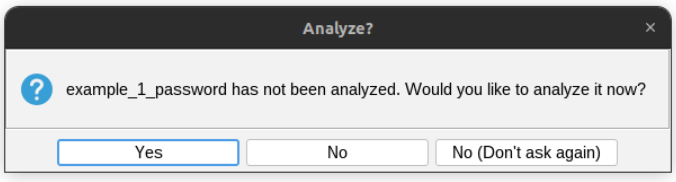


Fig. 10: Analyze prompt

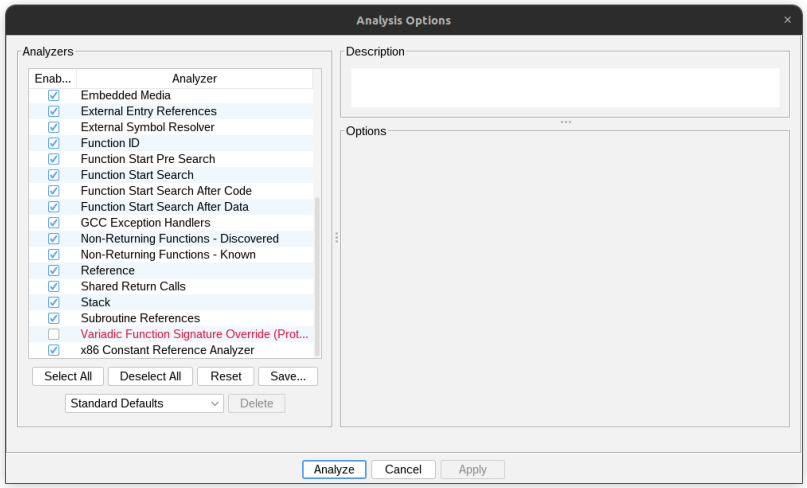


Fig. 11: Analyze prompt

3.1 Beginning analysis

Once the binary is imported, press the green dragon icon or double click the binary to open the main Ghidra window. When laucnhing a binary inside Ghidra for the first time, it will display a

prompt asking whether the binary should be analyzed. Click *Yes* then click *Analyze*. Do not change any of the settings Ghidra shows before pressing Analyze.

While analysis is in progress, Ghidra will display progress bars in the bottom right corner. Wait for them to disappear completely.

4 Exploring Ghidra Windows

Once we add the binary, we can start analyzing it with Ghidra. Ghidra provides a list of different analyzers that one can use.

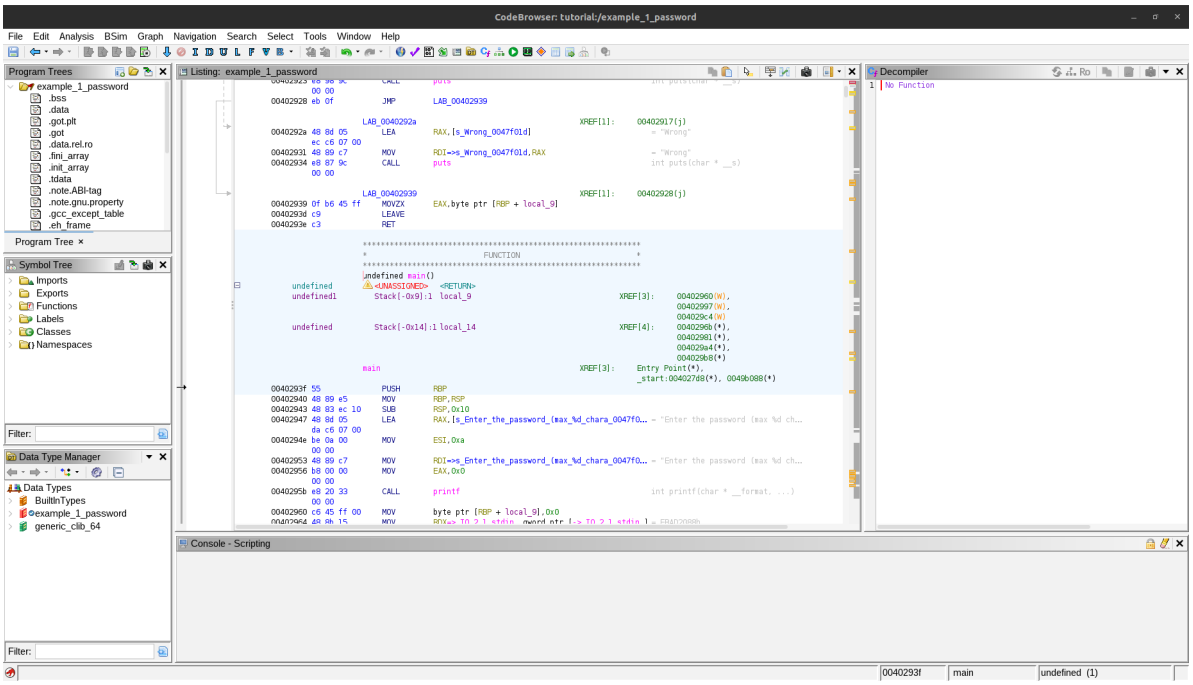


Fig. 12: Ghidra Main GUI

Once the previous setup steps are done, Ghidra will automatically open the Code browser. In the Code browser we can see several windows pertaining to our binary. Here is a breakdown of the windows that are numbered:

1. **Program Trees**: in this panel, we can see a breakdown of our ELF file binary segments. It is helpful to identify data regions and understand how the binary is loaded into memory.
2. **Symbol Tree**: Structure overview of the components in the binaries such as functions, classes, labels, variables, etc. It serves as a fast way to navigate to each area of the binary.
3. **Data Type Manager**: All data types that were captured during analysis will be organized and stored here. This includes primitive (int, double, float) types, structures, pointers, enums, etc. In this section, we will be able to manipulate data types if needed.
4. **Listing Window**: The core of our low-level analysis. It displays the disassembled machine instructions of the binary. From the leftmost side it lists the following:

```
00401833 48 8d 05        LEA        RAX,[s_Enter_the_password_(max_%d_chara_004980... = "Enter the password (max %d ch...
         ee 67 09 00
0040183a 48 89 c7        MOV        RDI=>s_Enter_the_password_(max_%d_chara_004980... = "Enter the password (max %d ch...
0040183d b8 00 00        MOV        EAX,0x0
         00 00
00401842 e8 e9 9e        CALL       printf                                            int printf(char * __format, ...)
```

Fig. 13: Snippet of Machine Code

The above line contains the following fields of information

- Address field → memory address where data is located (`0x00401833`)
- Bytes → opcode representing the instructions (`48 8d 05`)
- Mnemonic → instruction disassembled from the opcode (`LEA`)
- Instruction → machine instructions sometimes will show the output like in the picture above (`RAX,[s_Enter_ the_password_(max_%d_chara_004980...`)
- XREF → usually place at the beginning of each section header, generated by Ghidra to show other addresses referencing the current location or instructions.

```
XREF[3]:     Entry Point(*), 004000c0(*),
             _elfSectionHeaders::00000250(*)

XREF[4]:     Entry Point(*),
             _dl_map_object:0047e003(*),
             _dl_map_object:0047e01a(*),
             __sfp_handle_exceptions:00490cf5...
```

Fig. 14: XREF Header

5. **Decompile Window**: In this window we can see the generated C code Ghidra has created. It is worth mentioning that this generated C code typically does not exactly match the original source code of the binary during the reverse engineering process.
6. **Tool Bar**: Panel consists of various tools to aid analysis process, such as redo and undo.
7. **Scripting Window**: allows automation of task and customize analysis.

There are many other powerful panels listed under the **Windows** tab and we will skip the remaining ones for now.

5 Revealing Hard-Coded Password

5.1 Analyzing the Main Function

Once we are familiar with the user interface, it is time to go over our first Ghidra crack. The first thing to always start with is to look for the main function of the binary using the symbol tree. This is usually a great starting place for reverse engineering.

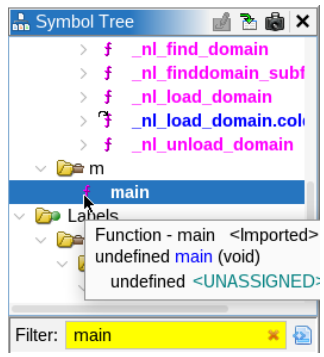
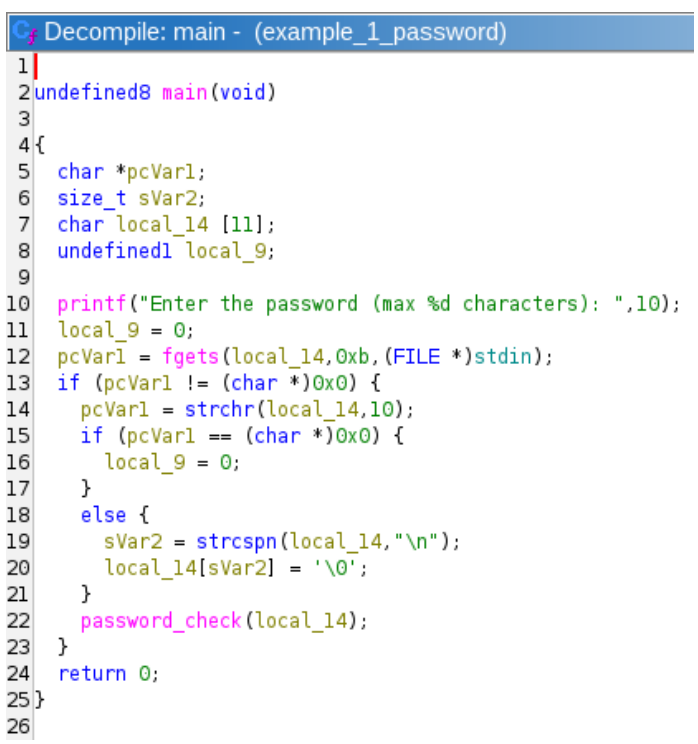


Fig. 15: Main function hovered in Symbol Tree Window

After double clicking on the main function, the Decompile window will show the main function code in C.



Decompile: main - (example_1_password)

```
1
2 undefined8 main(void)
3
4 {
5   char *pcVar1;
6   size_t sVar2;
7   char local_14 [11];
8   undefined1 local_9;
9
10  printf("Enter the password (max %d characters): ",10);
11  local_9 = 0;
12  pcVar1 = fgets(local_14,0xb,(FILE *)stdin);
13  if (pcVar1 != (char *)0x0) {
14    pcVar1 = strchr(local_14,10);
15    if (pcVar1 == (char *)0x0) {
16      local_9 = 0;
17    }
18    else {
19      sVar2 = strcspn(local_14,"\n");
20      local_14[sVar2] = '\0';
21    }
22    password_check(local_14);
23  }
24  return 0;
25 }
26
```

Fig. 16: Decompiled Code of function Main

The `main` function begins by declaring several local variables. The program prints "`Enter the password (max %d characters:`" with a max of 10 characters, then calls `fgets` to read up to 12 bytes from stdin into `local_14`. It then calls `strchr()` function to search for a newline character (`'\n'`) in the buffer; if one is found it is replaced with a null terminator `'\0'` to clearly strip the trailing newline from the input. The sanatized input in `local_14` is then passed directly to `password_check()` function, which we can assume handles the password verification logic.

5.2 Examining the password_check Function

Starting from the last function called `password_check`, we can examine its content by double clicking, which results in the code shown in the image below.

Decompile: password_check - (example_1_password)

```
1
2 bool password_check(char *param_1)
3
4 {
5   int iVar1;
6
7   iVar1 = strcmp(param_1,"pass");
8   if (iVar1 == 0) {
9     puts("Correct");
10  }
11  else {
12    puts("Wrong");
13  }
14  return iVar1 == 0;
15 }
16
```

Fig. 17: Decompiled code of function password_check

From this decompiled code, we can see that the function receives a char array as an input. It calls `strcmp()` function with the input comparing it to `"pass"`. The result of the comparison, `iVar1`, is then used to compare with 0. If yes, then the function will print out `"Correct"` indicating the user input is the same as the password. Otherwise, it is incorrect and will print out `"Wrong"`. The return value of this function is the updated value of variable `iVar1`.

5.3 Cleaning Up the Decompiled Code (Optional)

Using our observations, we can modify variable names to make the code more intuitive to look at, and likely more accurate/similar to the original source code. We can right click our variable name and press 'L' or right-click it and select `Rename Variable` or `Rename Function` for functions.

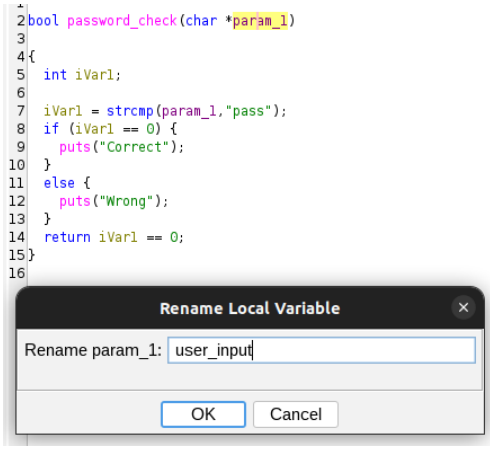


Fig. 18: Rename an function argument

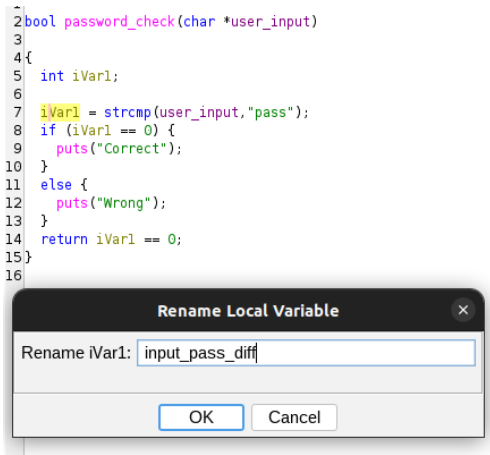


Fig. 19: Rename a variable

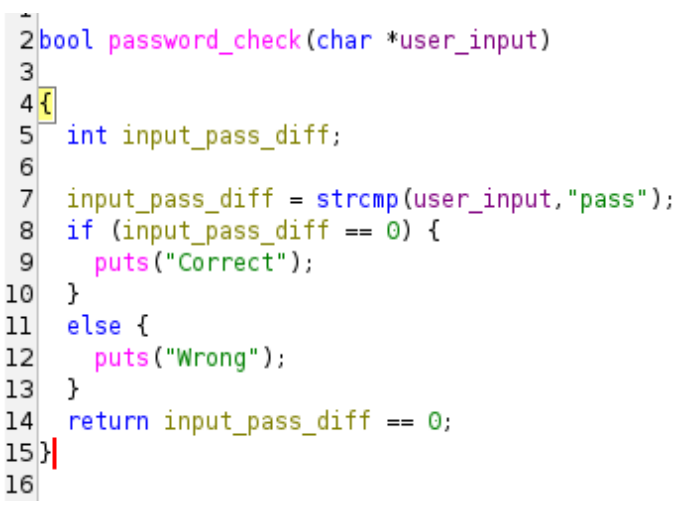


Fig. 20: Result in Cleaner Code (After chaning variable names)

and we can do the same for the variable storing the result from `strcmp`

After changing these variables, our code looks much cleaner. We can see that `user_input` and the constant `"pass"` are being compared with each other, it then checks if the result of that comparison is 0 for the "Correct" branch. In C, the `strcmp()` function receives two strings and returns an integer indicating which is "greater" based on the ASCII value of each character. If the function finishes comparing the final character and there is no difference, then the function returns 0 which means that the strings are equal. In other words, we can confirm that the hard-coded password is "pass" Given this observation, we can re-run the binary again and provide "pass" as the password, then we find that the program prints "Correct"

```
❯ ./example_1_password
Enter the password (max 10 characters): pass
Correct
```

Fig. 21: Binary Cracked with the correct password recovered using Ghidra

6 Patching the Code in Ghidra

In this section, we will try to modify the original binary and bypass the conditional jump instruction in the binary so that any input would be valid (i.e., printing "Correct"). Going into the password_check function, we can highlight the related machine code for the `if` statement, which makes the decision of whether a given input being correct. **In essence, we will generate a mutation of the original binary, where the functionality of the mutated binary will be different compared to the original one**.

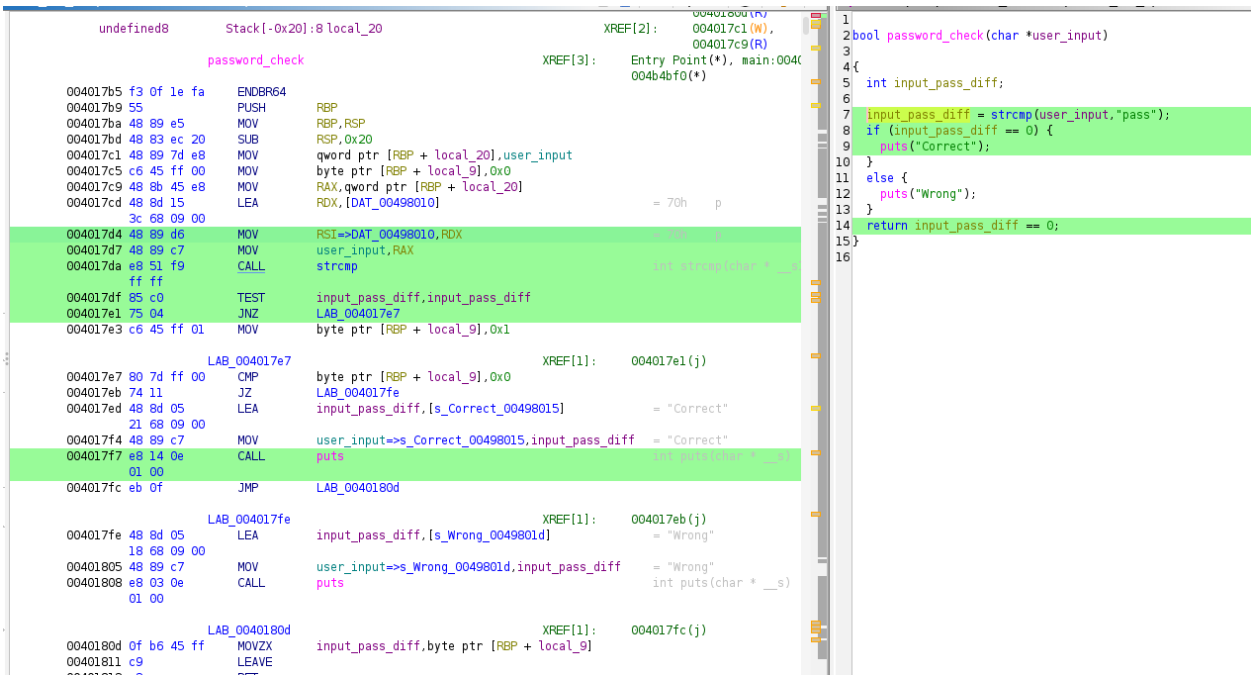


Fig. 22: Highlighted Code related to the Conditional Jump

In the figure above, we can examine that the function `password_check` is called at address `0x004017b5`. Specifically, it calls function `strcmp` at address `0x004017d4` and then the program compares whether the (renamed) variable `input_pass_diff` is zero by using a `test` instruction at address `0x004017df`. Then, at the next address `0x004017e1`, the program operates a `JNZ` (jump if NOT zero) instruction to jump to `LAB_004017e7`. If it does not jump, then at the next address `0x004017e3`, the program sets `RBP + local_9` to 0x1 by performing a `MOV` instruction. Next, we can see at `0x0041017e7` that it performs another `CMP` instruction to check if `RBP + local_9` is 0, and at address `0x00417eb` it uses `JZ` to either jump to `LAB_004017fe` where it prints "Wrong" OR continue linearly to address `0x00417f4` the next instruction, in which case it prints "Correct"

Now the question for us is, **what if we can do something to completely skip the jump even if the given password is not correct?** Then, as a result, the program can still print "Correct"

There are multiple ways to achieve this in this example. However, the simplest one is replacing the original jump if not zero instruction `JNZ` at address `0x004017e1` with NOP instructions `NOP`. The NOP instruction is a no-operation instruction in which the program tells the processor to do nothing and proceed to the next instruction in the stack. That way, the program always continues to `0x004017e3` where it sets `RBP + local_9` to 1 which leads to the program taking the "Correct" branch.

Specifically, we can right click on the line at address `0x004017e1` with bytes `75 04` (or instruction `JNZ`), and navigate to patch instructions. We replace the two bytes `75 04` with two bytes `90 90` where 0x90 is a NOP instruction. Note that a NOP instruction only has one byte while the original `jnz` instruction has two bytes. That is why we need to replace the original instruction with two NOP instructions after the padding to make sure the rest of the instructions in the binary remain unchanged.

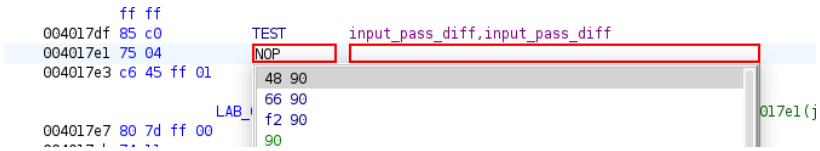


Fig. 23: Patch Instructions (before changes)

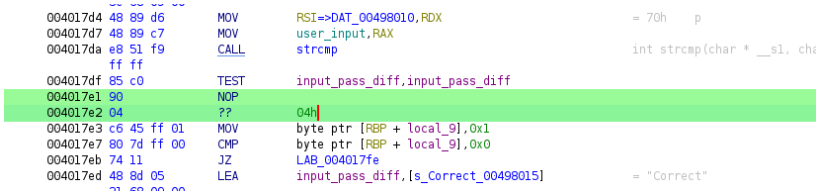


Fig. 24: Patch only the first byte (after the changes)

If we only modify the first byte with 90, then the second byte 04 is not decodable, which Ghidra will show with ?? at address `0x004017e2` for the second byte.

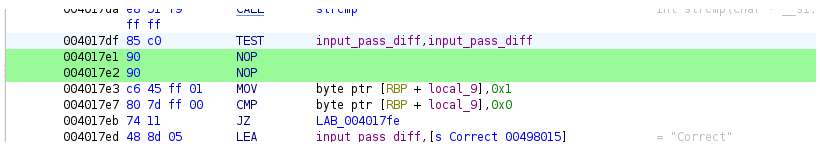


Fig. 25: Patch both bytes (after the changes)

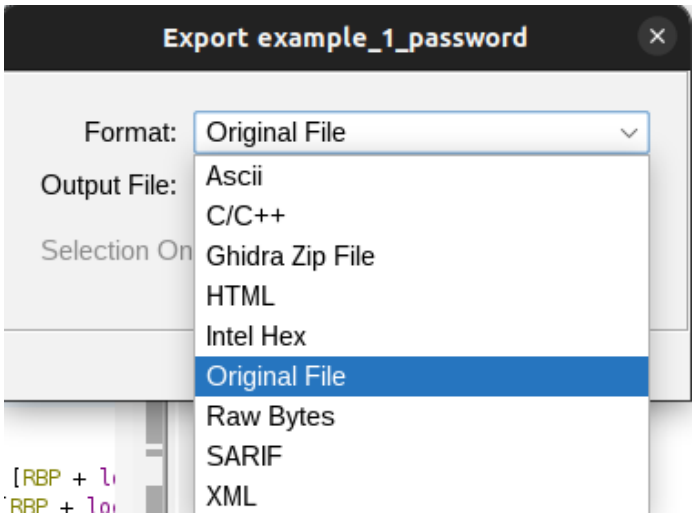


Fig. 26: Export Program: Format selection

Once the instruction line has been updated, we can export the modified binary and test it out. Click on **File** → **Export Program**, change file type to **original file**, and choose a name e.g. `example_1_patched`.

The last step is to verify that our patched binary will work as expected (i.e., passing given any password input). We provide an example below by using a random input 'sadhfasj" but the patched binary still prints out "Correct"

```
❯ ./example_1_patched
Enter the password (max 10 characters): sadhfasj
Correct
```

Fig. 27: Run example_1_patched with incorrect input

7 Reverse Engineering with Binary Ninja

In this section, we extend our tutorial by repeating the previous steps on the same binary (`example_1_password`) to identify the hard-coded password and patch the binary, but with another popular reverse engineering tool, Binary Ninja.

We assume we have the original binary from the previous steps. If not, we can re-run the command below to re-compile it.

```
$ gcc -static -o example_1_password example_1_password.c
```

7.1 Installing Binary Ninja

We use Binary Ninja 4.2, Commercial version (with an education discount)[2]. Binary Ninja is packaged either in an installer or a portable compressed folder. Binary Ninja should be installed using the given installer if provided. Otherwise, extract the contents of the compressed folder and open the binary.

7.2 Open the binary in Binary Ninja

Binary Ninja allows us to open a binary directly, this creates a single `.bndb` (binary ninja database) file in the same directory as the target binary. Press Open, we will now see a file explorer. Navigate to the correct folder and select the binary we just compiled (`example_1_password`) and open/double click the binary.

Alternatively, Binary Ninja has the option to a create project. A project can contain multiple binaries stored within a single `.bnpr` file. Projects allow us to place the database file anywhere, and are especially useful when working on multiple related binaries.

1. Press *New Project* → *pick any name, and any location (it does not have to share the directory with the binary)*
2. Press *Import Files* → *navigate to the folder containing the binary* → *select it* → *open*

Pick either option, the rest of the workflow will be the same regardless.

7.3 Exploring User Interface Windows in Binary Ninja

Once the binary is open in Binary Ninja, the user interface should look like the figure Fig. 28. There are 4 main components to the user interface in Binary Ninja:

1. **Symbol List**: Contains a list of the known program symbols

[2] https://binary.ninja/purchase

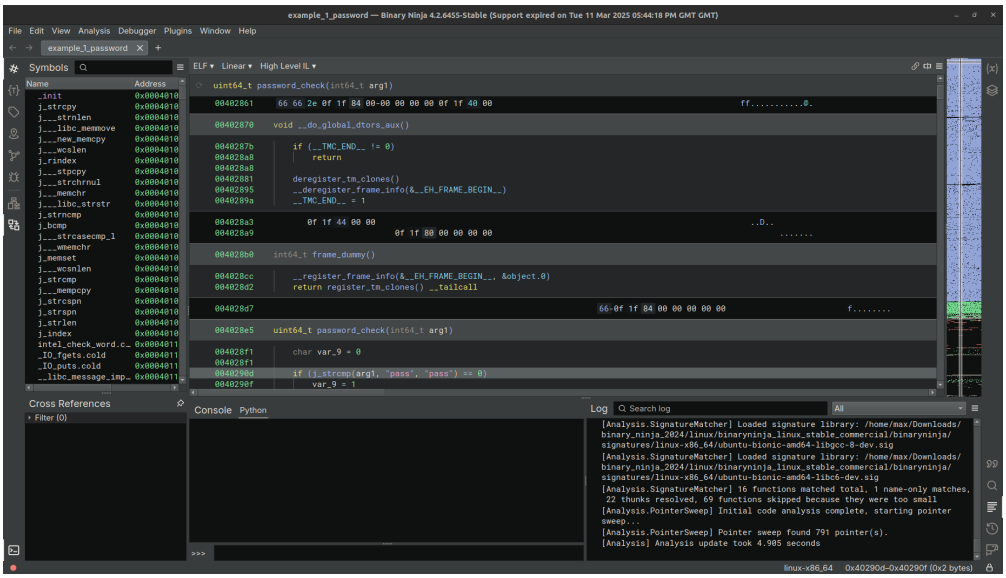


Fig. 28: User Interface in a Project

2. **Cross Reference (XREF)**: Contains reference for functions and memory. For example, a given function will have references to all of its callers and callees.
3. **Main View**: Shows the disassembly of the program
4. **Feature Map**: A heatmap of how Binary Ninja interprets the binary and how confident Binary Ninja is in the analysis.

7.4 Symbol List and Finding Main

Typically,we want to start our analysis from where user code execution begins which is (usually) the main function. To jump to the main function simply search or scroll to main in the symbol list.

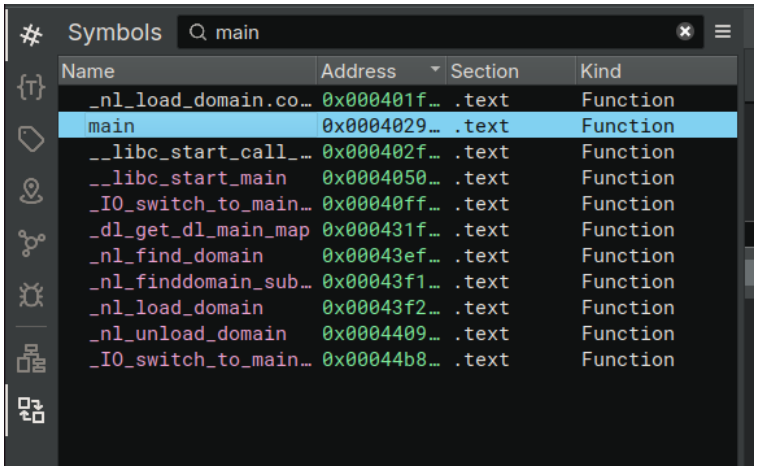


Fig. 29: Symbol list: finding main

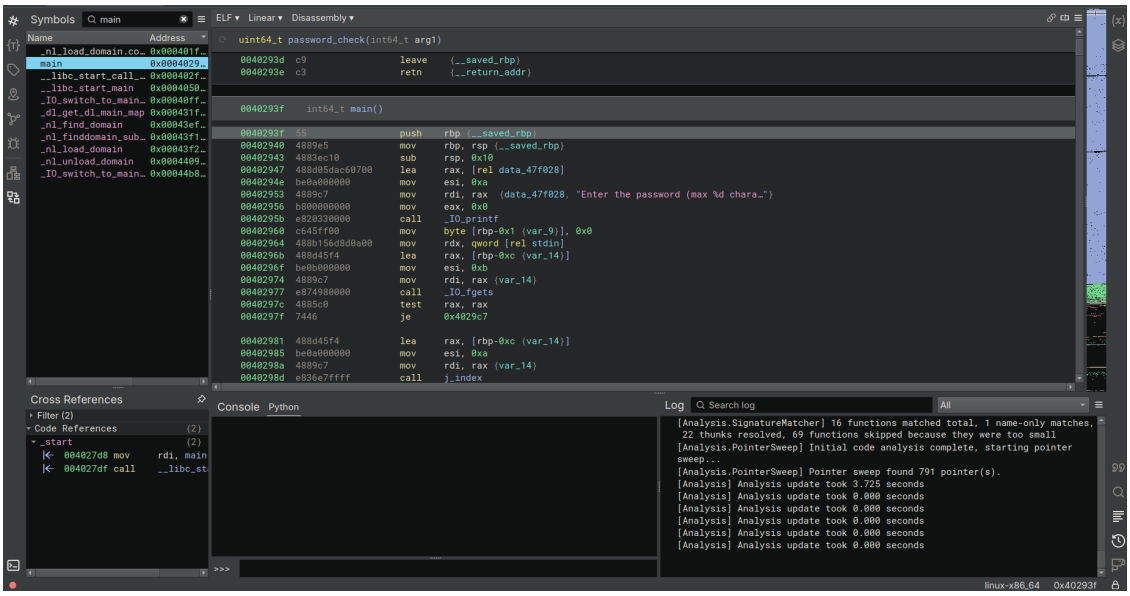


Fig. 30: View of the main function

After navigating to main, should be able to see the main function on your screen right now on the main window/pane

7.5 Layouts, Views, and Code Representation

At this point, we notice you have three UI elements, “ELF”, “Linear”, “Disassembly”, at the top left. All three are dropdowns that control how Binary Ninja displays the binary and code. The first one gives us the ability to switch between RAW and ELF mode. ELF is a binary file format used by Unix systems like Linux and BSD. For a Windows binary, it will shows ”PE” (Portable Executable) instead. Next, we can choose between different views such as “Byte Overview”, “Graph”, and the default, “Linear.” there are more options but most of the content related to this tutorial will be related to Linear or Graph view. Finally, we can observe “High Level IL”, this option controls how Binary Ninja presents the disassembled or decompiled code. Binary Ninja features a decompiler that produces multiple levels of IL (Intermediate Language). More information can be found here[3] It is recommended to utilize the split view to view disassembly and decompiler output side by side. For this tutorial we will use the pseudo-C code representation for our decompiler.

7.6 Examining the Program

Now that we have an understanding of navigating in Binary Ninja, let us examine the target binary. More particularly, the main function in the target binary `example_1_password`

[3] https://docs.binary.ninja/guide/index.html#high-level-il

```
0040293f    int64_t main()

0040293f    {
0040293f        _IO_printf("Enter the password (max %d chara…", 0);
00402960        char var_9 = 0;
0040297f        void var_14;
0040297f
0040297f        if (_IO_fgets(&var_14, 0xb, stdin))
0040297f        {
0040298d            if (j_index(&var_14, 0xa))
004029b3                *(uint8_t*)(&var_14 + j_strcspn(&var_14, &data_47f051, &data_47f051)) = 0;
00402995            else
00402997                char var_9_1 = 0;
00402997
004029c4            char var_9_2 = password_check(&var_14);
0040297f        }
0040297f
004029cd        return 0;
0040293f    }
```

Fig. 31: Close up view on main function pseudo-C code

From the highlighted line in the figure above we see that main function calls `password_check` function at address `004029c4`. We can double click on `password_check` to jump into the function and view its code. In the linear view of the `password_check` function, we can see similar logic recovered by Ghidra in the previous section. Specifically, `j_strcmp` function is called to compare `arg1` with a string "pass", and then update the value of variable `var_9` to 1 based on the result of this `j_strcmp` function call. If the value of this `var_9` is 0, then `_IO_puts` prints out "Wrong, otherwise, "Correct" In other words, we can infer "pass" is the hard-coded password we would like to reveal.

```
004028e5    uint64_t password_check(int64_t arg1)

004028e5    {
004028e5        char var_9 = 0;
004028e5
0040290d        if (!j_strcmp(arg1, "pass", "pass"))
0040290f            var_9 = 1;
0040290f
00402917        if (!var_9)
00402934            _IO_puts("Wrong");
00402917        else
00402923            _IO_puts("Correct");
00402923
0040293e        return (uint64_t)var_9;
004028e5    }
```

Fig. 32: password_check function: Linear view

Previously, we went over the different view settings Binary Ninja offers. Sometimes, it can be helpful to use the graph mode to understand and visualize branching logic. In the graph view, the code is split into blocks, and color-coded arrows are added as a guide. Arrows are only added when

branch instructions are present. - Green arrows follow the positive (true) case of an if statement - Red arrows follow the negative (false) case of an if statement - Blue arrows lead from the end of a branched block back to shared/common code blocks.

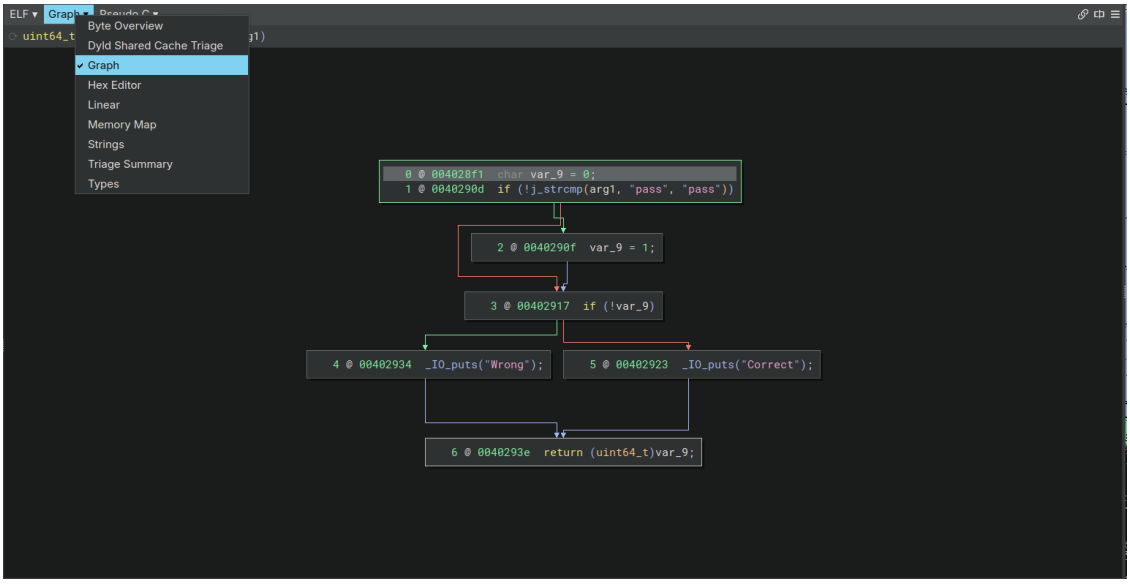


Fig. 33: password_check function: Graph view

7.7 Improving Decompiler Results (Optional)

At this point, we notice that we have a function call to `j_strcmp`. We know that strcmp is a function in C, but why Binary Ninja interpreters it as `j_strcmp`? In fact, Binary Ninja prepends the `j_` prefix to jump `thunks`. A thunk is a small piece of code whose sole purpose is to jump to the implementation of a given function located elsewhere. For example, if the program was dynamically linked `j_strcmp` would call `strcmp` from the `libc` shared object. These library call thunks are typically just a single `jmp` instruction.

More importantly, if we look closely at the call to `j_strcmp` in Fig. 32, we notice that Binary Ninja thinks it takes 3 arguments. This is not entirely correct, and is a common issue we will run into in the context of reverse engineering. It typically happens in complex code or unusual calling conventions. We can just ignore the third argument and consider it as a duplicate of the second argument, we then confirm the password is `"pass"`. It is also possible to manually correct the decompiler here with:

- Call site type override: only changes the function type for the current call site, so the function definition is unchanged. Think of it as adding a variable cast before a particular call expression.
- Change type: changes the variable/function type globally. To access either option, right click on the function or variable in the code. In our case, it does not matter which one we choose as

we only have a single call site. Typically, we will have to read the disassembly to rebuilt types and function signatures, but luckily `strcmp` is in the standard library so we can just look at the standard document to confirm it only has two arguments. As a result, we can update the function type as `int64_t j_strcmp(char*, char*)` as shown in Fig. 35.

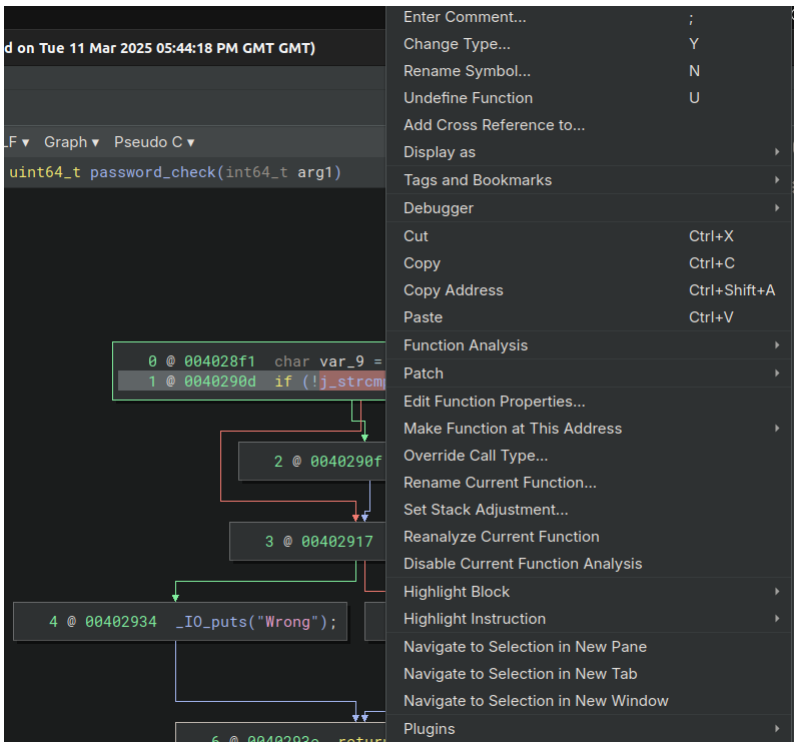


Fig. 34: Right click option/action menu

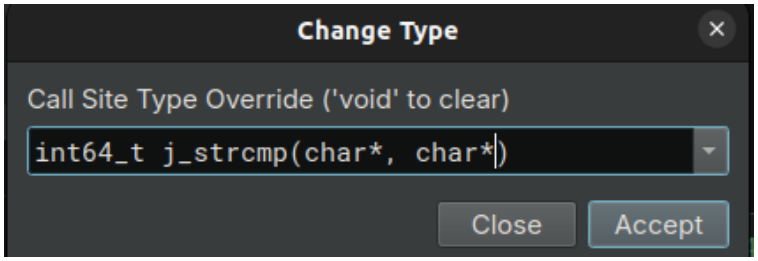


Fig. 35: Type textbox

We can then rename our variables (e.g., `var_9` to `is_correct`) and correct the return type (e.g., `uint64_t` to `bool`) for password_check in a similar way by using `Change Type` and `Rename Variable`. The code in Fig. 36 shows a more readable code after a few changes.

```
bool password_check(char* user_input)

{
    char is_correct = 0;

    if (!j_strcmp(user_input, "pass"))
        is_correct = 1;

    if (!is_correct)
        _IO_puts("Wrong");
    else
        _IO_puts("Correct");

    return is_correct;
}
```

Fig. 36: Decompiled code view after changes

7.8 Patching the Binary with Binary Ninja

Now, we would like to patch the binary similar as what we did with Ghidra such that the patched code will always print out "Correct" regardless of the user input. There are multiple ways to achieve this. In this example, we focus on the line `if (!j_strcmp(user_input, "pass"))` in the `password_check` function as this is the if statement controls whether the flag is_correct gets set, which decides the output of the program.

Typically, to patch an instruction in Binary Ninja we can open `Disassembly` view pane, select the instruction in assembly, and right click → Patch → Edit Current Line.

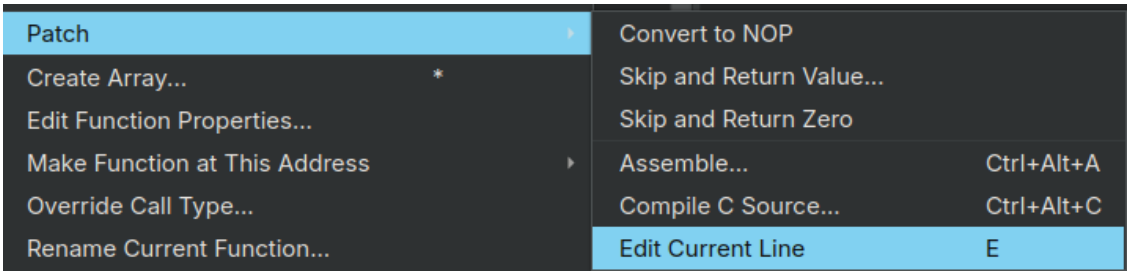


Fig. 37: Patch options on a call instruction

Binary Ninja makes things even easier for us. Specifically, it offers us different options/shortcuts depending on what kind of instruction we highlight. In our case, we want to patch a conditional jump instruction, finding it is as easy as clicking on the if statement in the decompiler view. We

do not even have to worry about the raw disassembly; If we right click the `if` statement and go to patch we get the shortcuts `Always branch`, `Never branch`, and `Invert branch`. Select `Never branch`. Once applied, we can observe the decompiled C code has changed accordingly and the assembly instructions are also changed due to this patching.

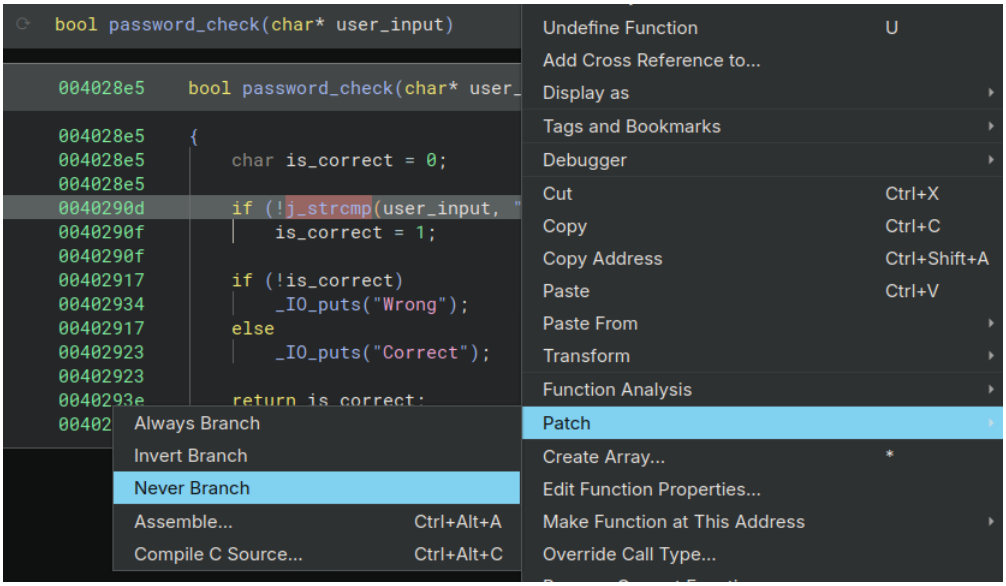


Fig. 38: Patching conditional instruction

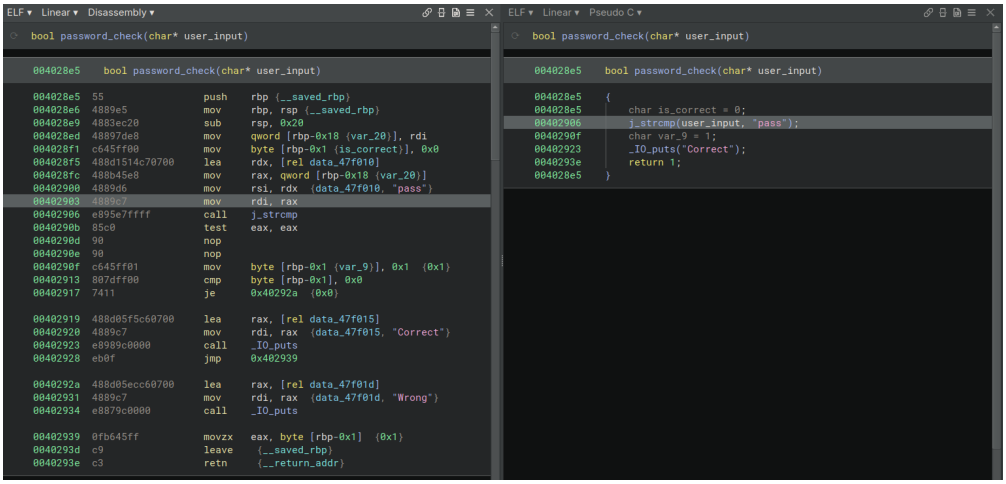


Fig. 39: Disassembly and Pseudo C (side-by-side) after patching

After this patching, we can now export the patched binary file and test it with a random password. To export the patched binary, we select "File→Save As", this will open a window. Choose a name for the patched binary, e.g. `password_1_patched`, and save. To run it in the terminal, first make the patched binary executable with the command `chmod +x password_1_patched`, then run

it with `./password_1_patched` . We should observe that any password will be accepted and show "Correct"

```
❯ ./example_1_patched
Enter the password (max 10 characters): sadhfasj
Correct
```

Fig. 40: Run `example_1_patched` with an incorrect input

8 Reverse Engineering with IDA

In this section, we repeat the previous steps on the same binary (`example_1_password.c`) to identify the hard-coded password and patch the binary, but with yet another popular reverse engineering tool, IDA. Build the binary with the command same as before:

```
$ gcc -static -o example_1_password example_1_password.c
```

8.1 Downloading and Installing IDA

IDA offers multiple editions including a free version called IDA Free. To obtain IDA Free, create an account on https://hex-rays.com/ida-free. If you do not receive an email with the license key after signing up, try to go onto the *My account* page and requesting a license key from there. After receiving the license, IDA Free will be available in the download center on the "My Account" page. Download the IDA installer and your license file (`.hexlic`). Then, we can run the following command to install

```
$ chmod +x ./ida-free-pc_93_x64linux.run
```

The name of the installer might be different depending on the edition and version of IDA you obtain. With IDA Free, you will not have access to Python scripting (IDAPY) and plugins included in more advanced version of IDA. Unfortunately, IDA Classroom is only available for professors rather than students. However, IDA Home can be used for non-commercial applications and can be purchased for less than $400. Note that these editions will only include a limited set of decompilers and will be cloud only, local decompilers are only available with expensive commercial licenses.

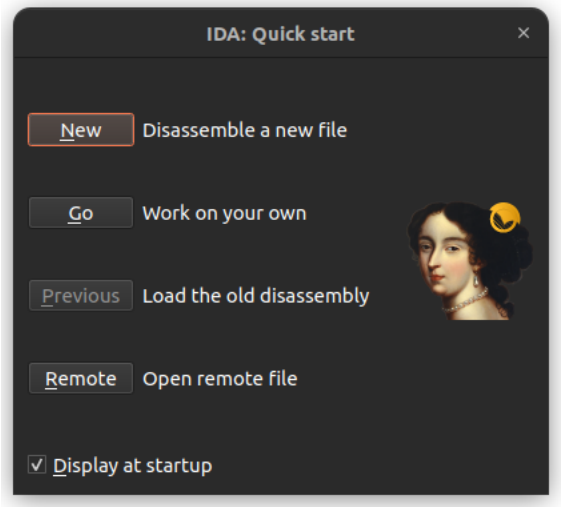


Fig. 41: Quick start window (IDA Free)

8.2 Opening the Binary in IDA

When we first open IDA, we will be greeted by a quick start window. Select *New*, if we have never used IDA before, then we will probably get an error complaining about missing license, continue through the popups until we get to the license manager. We will then want to press the three dots and find the license we downloaded earlier.

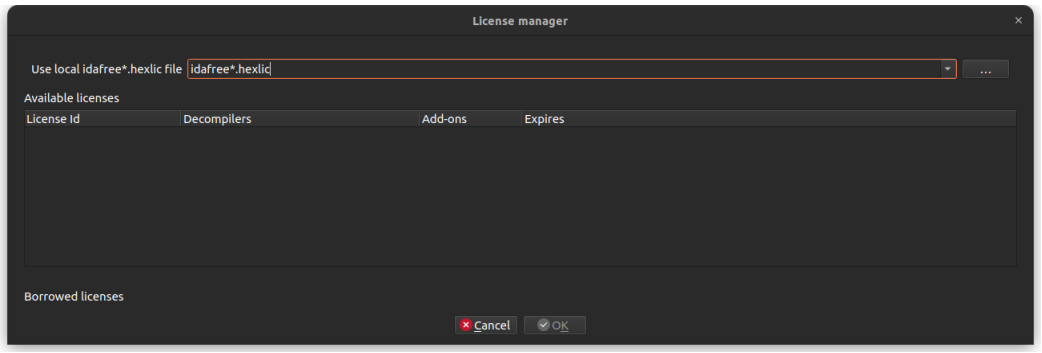


Fig. 42: License Manager by Indiciating the `.hexlic` File

Next, we can load the binary file, in our case, `example_1_password` by navigating to *File* → *Open*.

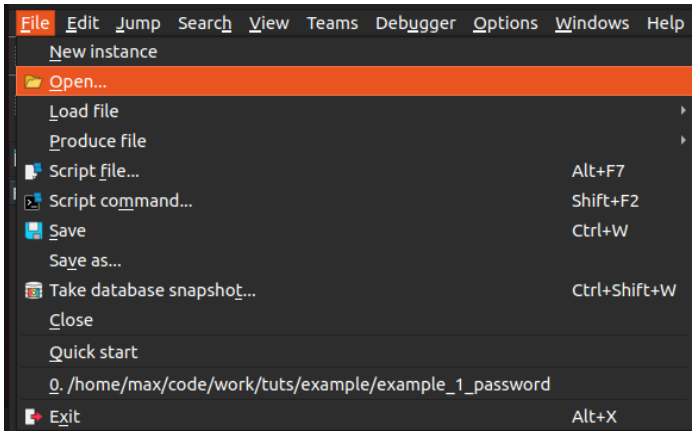


Fig. 43: Open File

8.3 Examining Disassembly and Decompiled Code (Subviews)

Initially, IDA launches with an "IDA View" subview which shows the disassembly of the target binary in the graph view. We can switch between the graph view and text view by clicking anywhere within the window. However, trying to reverse a program with only the disassembly can be difficult. In addition to IDA itself, we have access to the hex-rays decompiler. The decompiler is integrated

in IDA in the form of a pseudocode subview. We can open a pseudocode subview by pressing *F5* inside the IDA View subview or by going to *View* → *Open subviews* → *Generate pseudocode*. *Note: when we use a decompiler for the first time we might have to go through multiple pop-ups pressing OK/Agree*

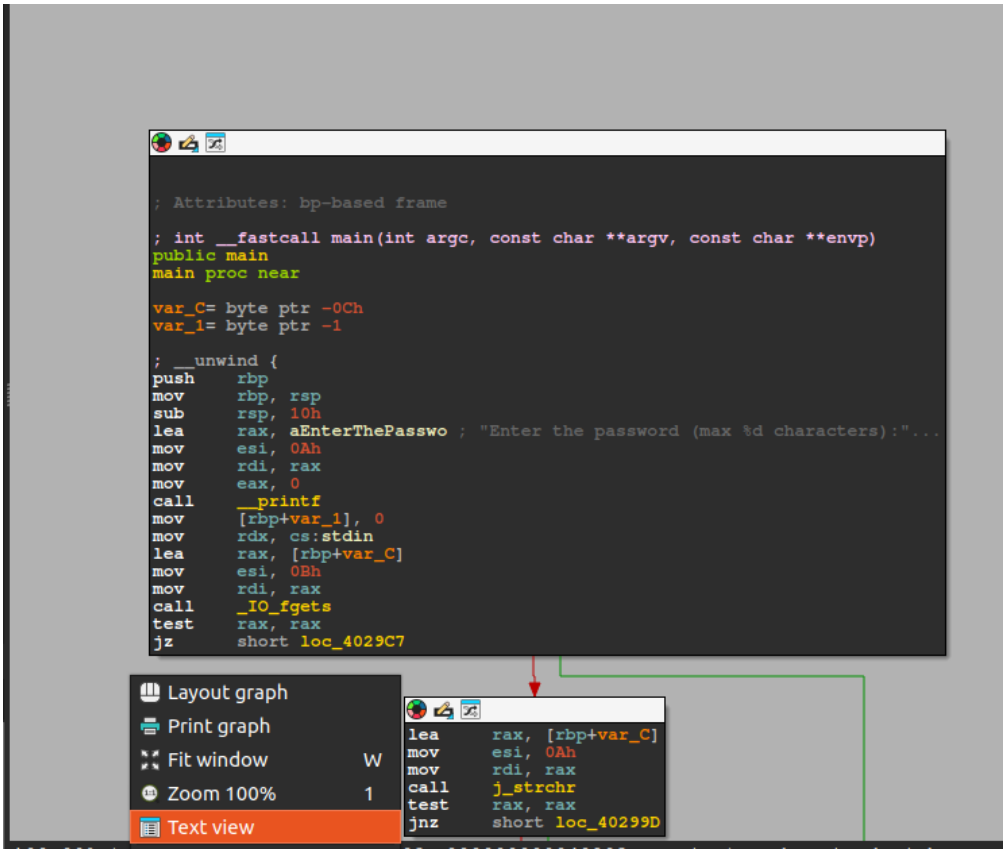


Fig. 44: Switch to textview

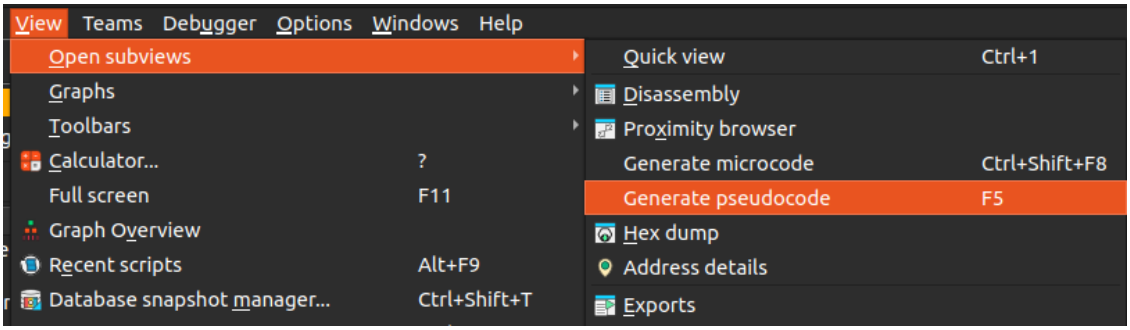


Fig. 45: Generate pseudocode

A neat feature in IDA is *window docking*. All windows and subviews in IDA can be dragged and docked so that we can customize the layout. IDA refers to layouts as "desktops" and we can save/reset these under *Windows* at the top. We will dock the pseudocode subview to be right beside our disassembly (IDA View) subview.

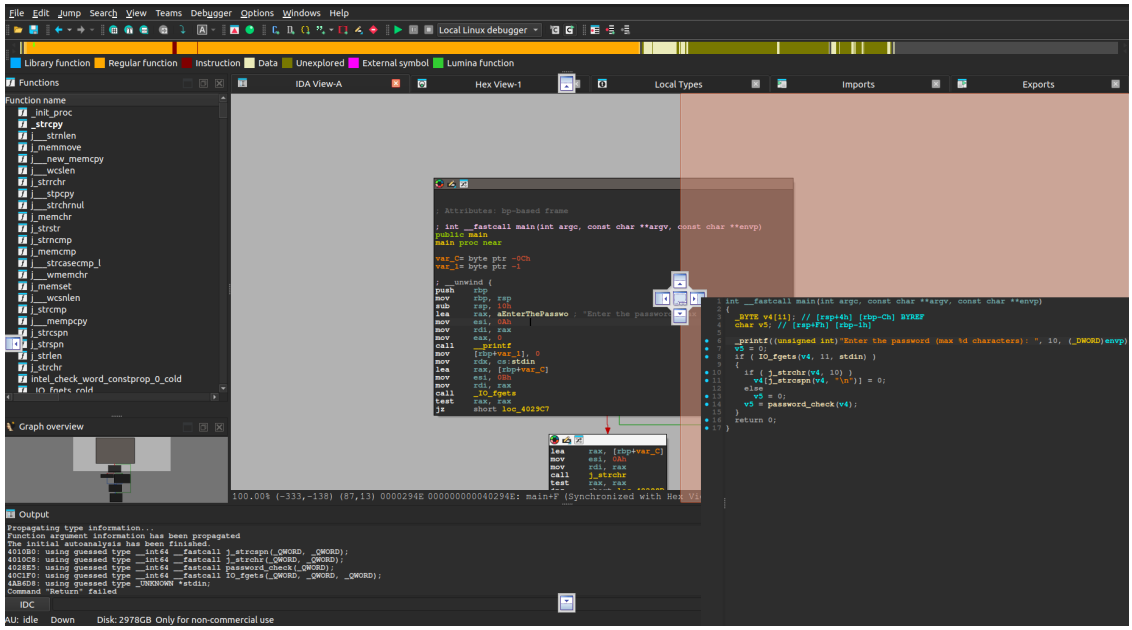


Fig. 46: Docking pseudocode subview

Now that the subviews are side-by-side *Right Click* → *Synchronize with* → *Pseudocode-A*. This will allow us to highlight assembly instructions to each line of code in the pseudocode, which will facilitate our analysis.

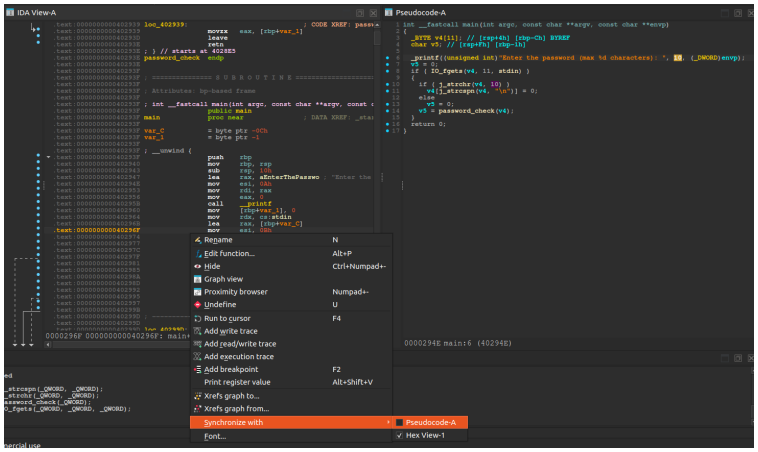


Fig. 47: Synchronize views

8.4 Revealing Hard-Coded Password with IDA

We can find the *main* function of this target binary via the functions subview on the left. To search for a function, click any function within the list followed by *Ctrl + F*. If we are not in *main* function, search for main and double click it to open it.

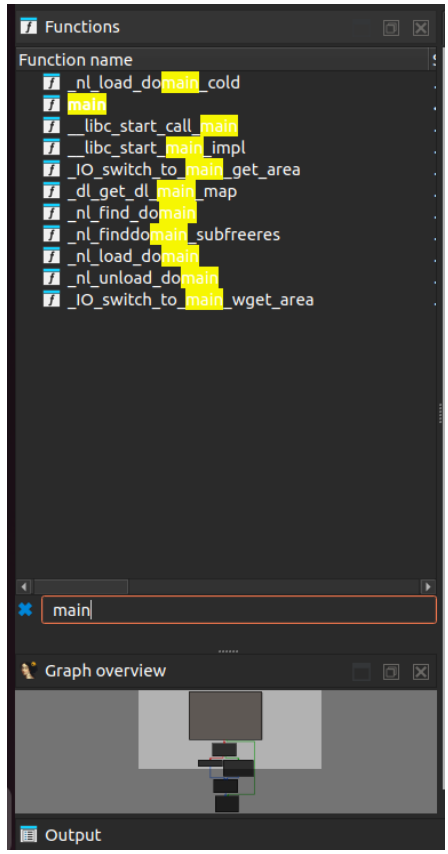


Fig. 48: Search functions

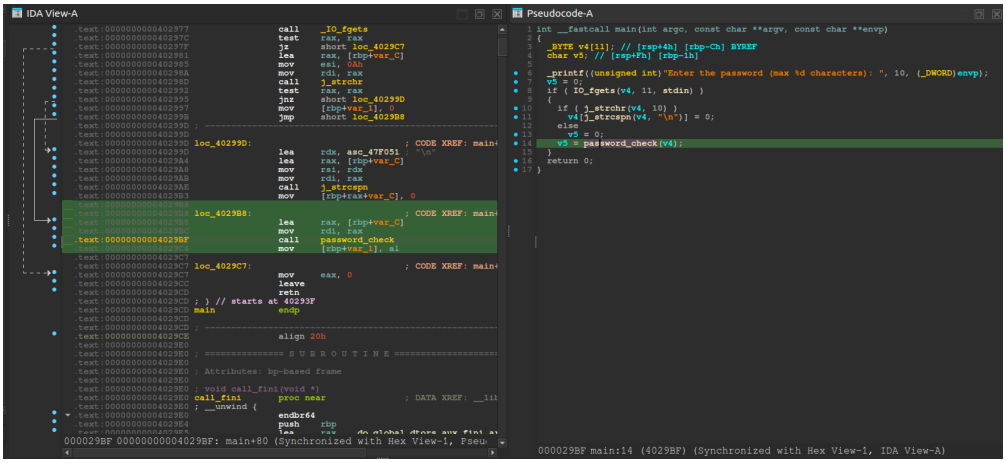


Fig. 49: Function main

Once inside the `main` function, we should be able to find a call to *password_check* function, and we can double click on this function to open it. In the pseudocode view at line 5, we can observe that a variable `a1` is compared with a string "pass", and the result of this comparison is assigned to a variable `v2`. Then, the program will output "Correct" or "Wrong" based on this `v2`. Given this logic, we can infer that "pass" is the hard-coded password in this target binary.

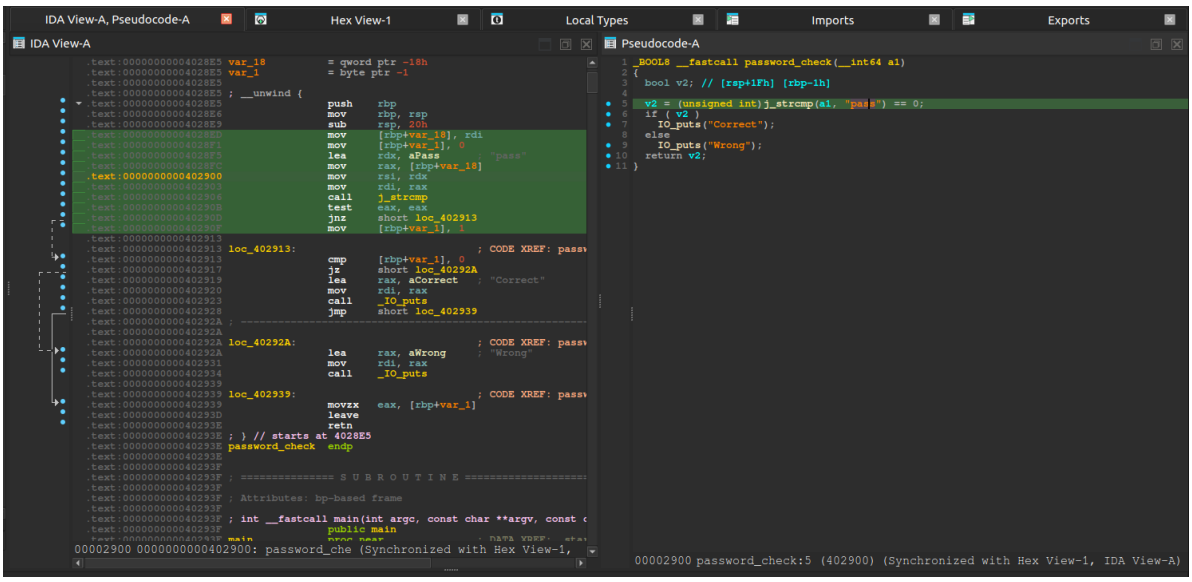


Fig. 50: Function password_check

Similar as Ghidra and Binary Ninja, IDA includes features to rename function names and variables in the pseudo code to make it more readable as shown in Fig. 51

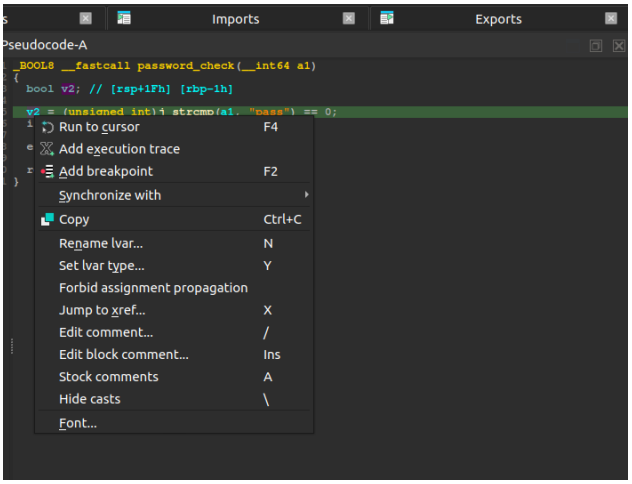


Fig. 51: Function right click variable options

```
Pseudocode-A
1  _BOOL8 __fastcall password_check(const char *user_inp
2  {
3    bool is_correct; // [rsp+1Fh] [rbp-1h]
4
5    is_correct = j_strcmp(user_input, "pass") == 0;
6    if ( is_correct )
7      IO_puts("Correct");
8    else
9      IO_puts("Wrong");
10   return is_correct;
11 }
```

Fig. 52: Function password_check (after renaming variable names)

8.5 Patching the Binary with IDA Free

Without plugins from the commercial version, patching can be a little challenging for a beginner in IDA. It is not as simple as clicking a conditional check and choosing a patching option as in Binary Ninja. We will have to examine the raw disassembly, patch the target binary instruction by instruction, and if any, handle the instruction size differences, which is the fun of software reverse engineering.

```
IDA View-A, Pseudocode-A | Hex View-1 | Local Types | Imports | Exports
IDA View-A
.text:00000000004028E5 var_18          = qword ptr -18h
.text:00000000004028E5 var_1           = byte ptr -1
.text:00000000004028E5
.text:00000000004028E5 ; __unwind {
.text:00000000004028E5                 push    rbp
.text:00000000004028E6                 mov     rbp, rsp
.text:00000000004028E9                 sub     rsp, 20h
.text:00000000004028ED                 mov     [rbp+var_18], rdi
.text:00000000004028F1                 mov     [rbp+var_1], 0
.text:00000000004028F5                 lea     rdx, aPass      ; "pass"
.text:00000000004028FC                 mov     rax, [rbp+var_18]
.text:0000000000402900                 mov     rsi, rdx
.text:0000000000402903                 mov     rdi, rax
.text:0000000000402906                 call    j_strcmp
.text:000000000040290B                 test    eax, eax
.text:000000000040290D                 jnz     short loc_402913
.text:000000000040290F                 mov     [rbp+var_1], 1
.text:0000000000402913
.text:0000000000402913 loc_402913:                             ; CODE XREF: pass
.text:0000000000402913                 cmp     [rbp+var_1], 0
.text:0000000000402917                 jz      short loc_40292A
.text:0000000000402919                 lea     rax, aCorrect   ; "Correct"
.text:0000000000402920                 mov     rdi, rax
.text:0000000000402923                 call    _IO_puts
.text:0000000000402928                 jmp     short loc_402939
.text:000000000040292A ; ---------------------------------------------------------------
.text:000000000040292A
.text:000000000040292A loc_40292A:                             ; CODE XREF: pass
.text:000000000040292A                 lea     rax, aWrong     ; "Wrong"
.text:0000000000402931                 mov     rdi, rax
.text:0000000000402934                 call    _IO_puts
.text:0000000000402939
.text:0000000000402939 loc_402939:                             ; CODE XREF: pass
.text:0000000000402939                 movzx   eax, [rbp+var_1]
.text:000000000040293D                 leave
.text:000000000040293E                 retn
.text:000000000040293E ; } // starts at 4028E5
.text:000000000040293E password_check  endp
.text:000000000040293E
.text:000000000040293F
.text:000000000040293F ; =============== S U B R O U T I N E =======================
.text:000000000040293F
.text:000000000040293F ; Attributes: bp-based frame
.text:000000000040293F
.text:000000000040293F ; int __fastcall main(int argc, const char **argv, const c
.text:000000000040293F                 public main
00002900 0000000000402900: password_che (Synchronized with Hex View-1,

Pseudocode-A
1  _BOOL8 __fastcall password_check(__int64 a1)
2  {
3    bool v2; // [rsp+1Fh] [rbp-1h]
4
5    v2 = (unsigned int)j_strcmp(a1, "pass") == 0;
6    if ( v2 )
7      IO_puts("Correct");
8    else
9      IO_puts("Wrong");
10   return v2;
11 }

00002900 password_check:5 (402900) (Synchronized with Hex View-1, IDA View-A)
```

Fig. 53: Function password_check before patching

Since the (renamed) variable *is_correct* (as shown in Fig. 52) controls what the program outputs, we can select line 5 in the pseudocode view where its value is assigned. On the left side in IDA View, we can see that a `jnz` instruction ad address `0x40290D` decides whether variable `is_correct`

(`[rbp + var_1]` in assembly) is set to 1 (true) by a `mov` instruction at address `0x40290F` or whether to jump over to address `0x402913` and set it to 0 (false). Given this logic, we can replace this `jnz` instruction with NOP instruction, which will always make the value of variable `is_correct` (`[rbp + var_1]` in assembly) as 1.

To do this, we can click on the `test` instruction above the `jnz` instruction. Then, we go to *Edit* → *Patch program* → *Assemble*, and replace the instruction with *NOP*. After clicking Ok, the window will stay open. We can simply move to the `jnz` instruction. We will need to provide `4` NOPs in total, so we need to repeat the process `4` times before we close the window. This is because the `test` and `jnz` instructions we would like to replace have a total of 4 bytes and each NOP only has 1 byte.

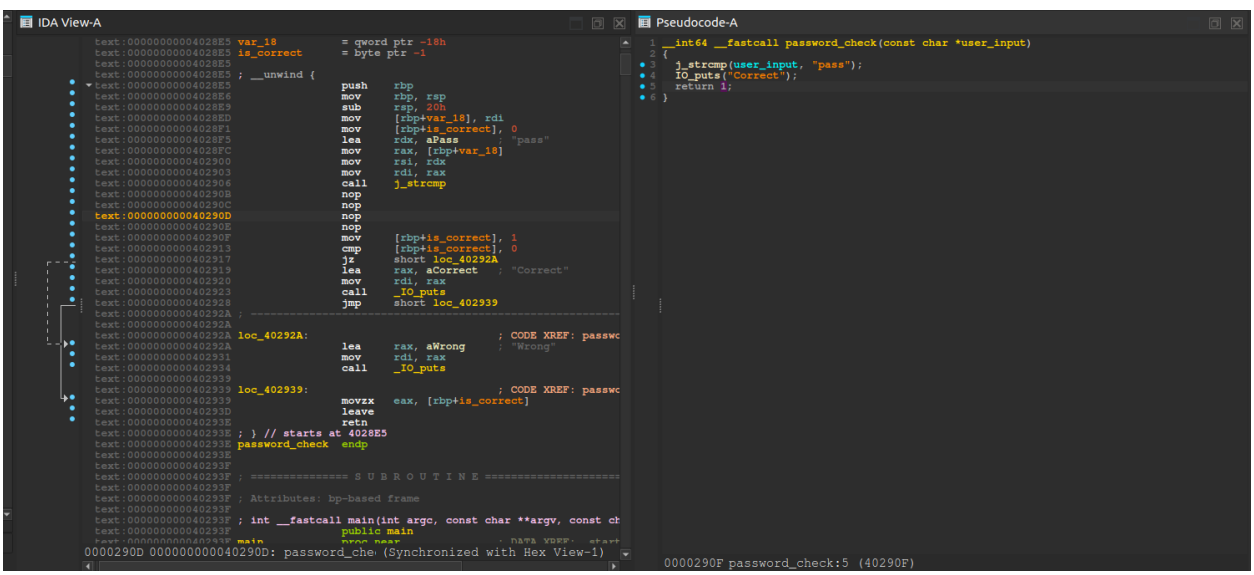


Fig. 54: Function password_check after patching

To apply the patches (i.e., exporting the modified binary), we can go to *Edit* → *Patch program* → *Apply patches to input file...* Unlike Ghidra and Binary Ninja, IDA will apply the modifications to the original binary instead of saving a patched copy with a new file name, so make sure to keep a copy of the original binary by checking *Create backup* before you click *OK*. By default, Create backup will create a copy by the same name in the same location as the original and append a `.bak` suffix to it.

If the patches are applied successfully, we will see "Applied 4/4 patch(es)" printed in the output subview at the bottom. Otherwise, make sure your file is not open and is not being used by any other programs and try again.

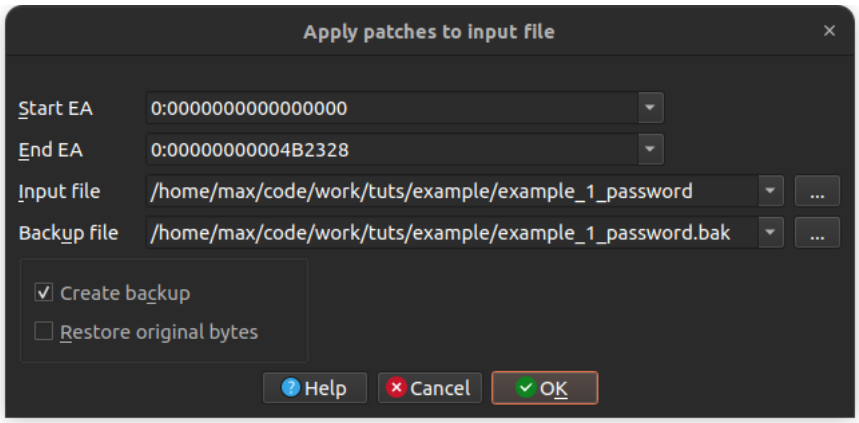


Fig. 55: Apply patches window

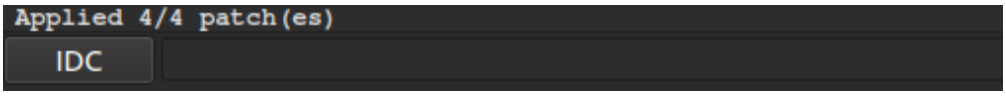


Fig. 56: Apply patches success

Once the patching is successful, we should can run the patched binary and enter anything input to get "Correct"

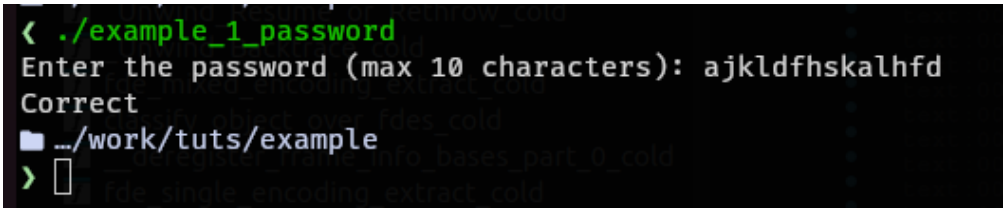


Fig. 57: Result of running the patched binary

9 Conclusion

In conclusion, we are able to identify, analyze, and modify our first binary using Ghidra's reverse engineering capabilities. This example demonstrates the fundamental techniques for static analysis, identifying call functions, and binary patching. The workflow presented here provides a foundation for more difficult tasks in reverse engineering along the way. In addition, we also show how to achieve similar objectives by using other popular reverse engineering tools, including Binary Ninja and IDA (Free).

There are many resources online. We only list a few at the end of this document [1–4] in case one would like to explore more resources related to (software) reverse engineering. Besides software reverse engineering, hardware reverse engineering is also a critical topic and even harder to learn. We do encourage students to explore hardware reverse engineering as well.

References

1. https://crackmes.one/.
2. https://docs.binary.ninja/guide/.
3. https://github.com/nationalsecurityagency/ghidra.
4. https://picoctf.org/about.html.